

1장. 웹과 프론트엔드 시작하기

학습 목표 - 웹의 클라이언트-서버 구조와 HTTP 요청-응답 흐름을 설명할 수 있습니다
- HTML, CSS, JavaScript 각각의 역할을 구분하여 이해할 수 있습니다 - VS Code와 Live Server로 개발 환경을 구축할 수 있습니다 - 첫 번째 HTML 페이지를 만들어 브라우저에서 실행할 수 있습니다

1.1 웹은 어떻게 동작하는가

도입

웹 페이지를 만들기 전에, 먼저 웹이 어떻게 동작하는지 이해해야 합니다. 주소창에 `https://www.naver.com` 을 입력하고 엔터를 누르는 순간 실제로 어떤 일이 벌어지는지 생각해 본 적이 있습니까? 이 절에서는 그 과정을 단계별로 살펴보고, 우리가 만들 코드가 어떤 역할을 하는지 이해합니다.

핵심 개념 설명

인터넷과 웹의 차이 (Internet vs. Web)

인터넷(Internet)과 **웹(Web)**은 자주 혼용되지만 서로 다른 개념입니다.

인터넷은 전 세계 컴퓨터들이 연결된 거대한 통신 네트워크 인프라입니다. 쉽게 말해, 건물과 건물을 이어 주는 도로망과 같습니다. 웹은 그 도로 위에서 운행하는 버스나 택시처럼, 인터넷 위에서 동작하는 하나의 서비스입니다.

인터넷이 만들어진 것은 1960년대이고, 웹은 1989년 팀 버너스리(Tim Berners-Lee)가 고안했습니다. 전자메일, FTP, 웹 등 여러 서비스가 인터넷을 기반으로 동작합니다.

정리: 인터넷은 도로망, 웹은 그 위에서 달리는 버스입니다.

클라이언트-서버 구조 (Client-Server Architecture)

웹의 동작은 **클라이언트(Client)**와 **서버(Server)** 두 역할로 나뉩니다.

- **클라이언트**: 웹 페이지를 요청하는 쪽입니다. 우리가 사용하는 웹 브라우저(Chrome, Edge, Firefox 등)가 클라이언트 역할을 합니다.
- **서버**: 요청을 받아 페이지 파일을 **응답**으로 돌려주는 컴퓨터입니다.

음식점에 비유하면 이렇습니다. 손님(클라이언트)이 메뉴를 주문하면 주방(서버)이 음식을 만들어 내줍니다. 손님은 음식 만드는 방법을 몰라도 됩니다. 결과물만 받으면 됩니다.

오해 바로잡기: 서버는 거대한 특수 장비가 아닙니다. 내 컴퓨터도 서버 소프트웨어를 실행하면 서버가 됩니다. Live Server 확장을 설치하면 여러분의 컴퓨터가 바로 서버 역할을 합니다.

주소창에 URL을 입력하면 일어나는 일

URL(Uniform Resource Locator)은 인터넷 상의 자원 위치를 나타내는 주소입니다.

`https://www.example.com/about.html` 형태로 생겼습니다.

URL을 입력하고 엔터를 누르면 다음 순서로 진행됩니다.

1. 브라우저가 URL을 분석하여 서버의 주소를 찾습니다 (DNS 조회).
2. 브라우저가 서버에 **HTTP(S) 요청(Request)**을 보냅니다.
3. 서버가 요청을 처리하여 HTML, CSS, JS 파일을 **응답(Response)**으로 돌려줍니다.
4. 브라우저가 받은 파일을 분석하여 화면에 그립니다.

HTTP 요청-응답 흐름 (클라이언트-서버 구조)



요청-응답 과정 요약

1단계: 사용자가 주소창에 URL 입력
2단계: 브라우저가 서버에 HTTP 요청 전송

3단계: 서버가 HTML-CSS-JS 파일로 응답
4단계: 브라우저가 파일을 해석해 화면 렌더링

HTTP 요청-응답 흐름 (클라이언트-서버 구조)

요청과 응답: HTTP 이해하기

HTTP(HyperText Transfer Protocol)는 클라이언트와 서버가 데이터를 주고받는 규칙(프로토콜)입니다. 마치 택배 회사가 정한 포장 규격과 배송 절차처럼, 데이터를 어떤 형식으로 주고 받을지 정해 둔 약속입니다.

요청에는 `GET /index.html` 같은 형식으로 "이 파일을 보내달라"는 내용이 담깁니다. 응답에는 `200 OK` 같은 **상태 코드(Status Code)**와 함께 파일 내용이 담겨 옵니다.

상태 코드	의미
200 OK	요청 성공, 파일을 응답으로 보냄
404 Not Found	요청한 파일이 서버에 없음
500 Internal Server Error	서버 내부 오류

코드로 확인하기

다음은 나중에 직접 만들 HTML 파일이 어떤 형태로 브라우저에 전달되는지 이해하기 위해, 가장 단순한 HTML 응답 예시를 살펴보는 코드입니다.

```
<!-- 브라우저가 서버로부터 받는 응답의 내용 (가장 단순한 형태) -->
<!DOCTYPE html>
<html lang="ko">
  <head>
    <meta charset="UTF-8">
    <title>나의 첫 페이지</title>
  </head>
  <body>
    <h1>안녕하세요, 웹 세계에 오신 것을 환영합니다!</h1>
  </body>
</html>
```

브라우저 화면에는 다음과 같이 표시됩니다.

안녕하세요, 웹 세계에 오신 것을 환영합니다! ← 가장 큰 제목 형태로 표시

이 HTML 파일 하나가 서버에서 브라우저로 전달되는 것이 웹의 핵심입니다.

절 요약

- 인터넷은 통신 인프라이고, 웹은 그 위에서 동작하는 서비스입니다.
- 브라우저(클라이언트)가 서버에 HTTP 요청을 보내면, 서버가 HTML 파일을 응답으로 보냅니다.
- URL은 서버 위의 자원 주소이며, HTTP는 주고받는 규칙(프로토콜)입니다.

1.2 브라우저와 렌더링 과정

도입

서버에서 받은 파일들을 브라우저가 어떻게 아름다운 화면으로 바꾸는지 궁금하지 않습니까? 이 절에서는 웹 프론트엔드를 이루는 세 가지 언어인 HTML, CSS, JavaScript의 역할을 명확히 구분하고, 브라우저가 코드를 화면으로 그리는 과정을 이해합니다.

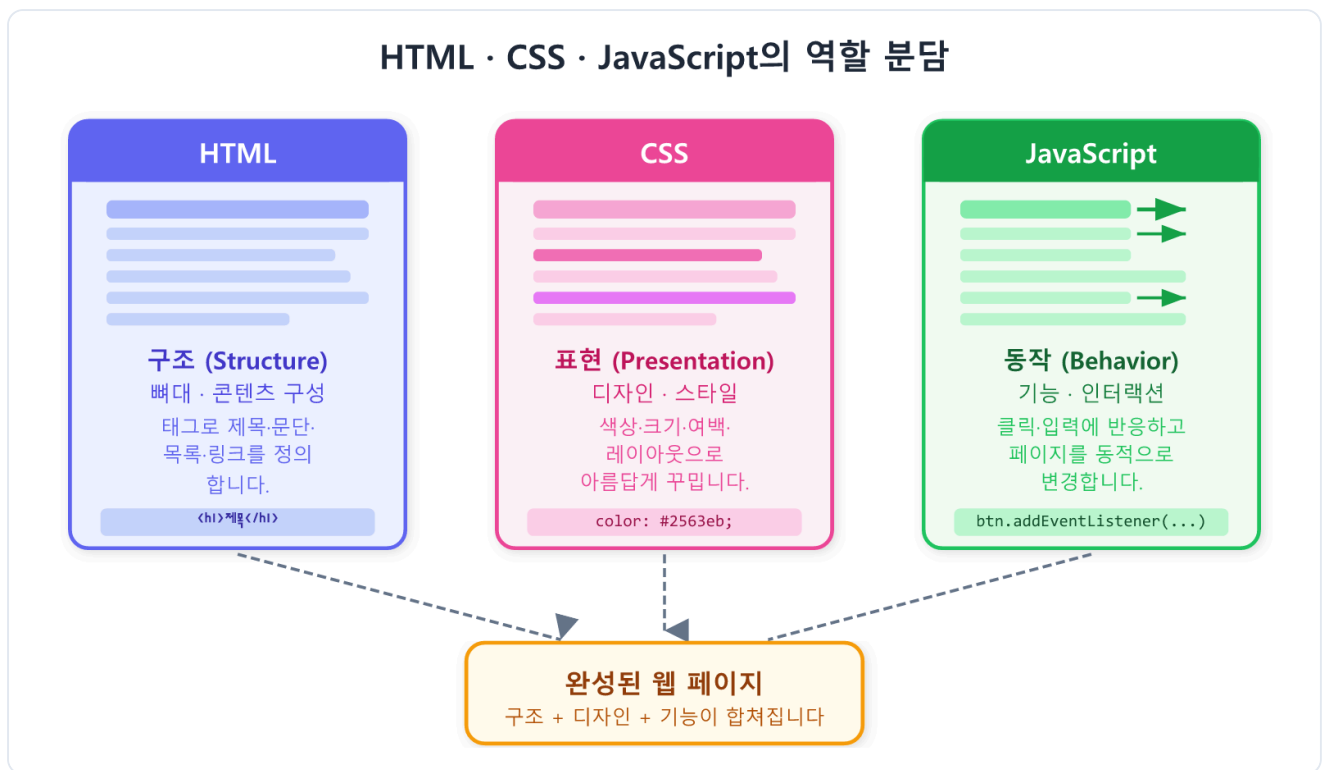
핵심 개념 설명

프론트엔드를 이루는 세 가지 언어

웹 페이지 하나는 세 가지 언어가 협력하여 만들어집니다. 각 언어는 담당 역할이 명확하게 분리되어 있습니다.

집을 짓는 데 비유하면 이렇습니다.

- **HTML**: 집의 뼈대와 구조입니다. 방이 몇 개인지, 창문은 어디에 있는지를 정의합니다.
- **CSS**: 인테리어와 페인트입니다. 벽 색깔, 가구 배치, 전체적인 분위기를 결정합니다.
- **JavaScript**: 전기 배선과 스위치입니다. 전등을 켜고 끄거나 문을 자동으로 여닫는 동작을 담당합니다.



HTML · CSS · JavaScript의 역할 분담

HTML은 구조 (Structure)

HTML(HyperText Markup Language)은 웹 페이지의 뼈대를 만드는 언어입니다. 제목, 단락, 이미지, 버튼 등 페이지에 들어갈 요소들을 정의합니다.

HTML만으로는 특별한 디자인이 없는 텍스트 위주의 문서가 됩니다. 하지만 내용(콘텐츠)의 구조와 의미는 HTML이 담당합니다.

CSS는 표현 (Presentation)

CSS(Cascading Style Sheets)는 HTML 요소의 모양을 꾸미는 언어입니다. 글자 색, 배경색, 크기, 여백, 배치 방식 등을 지정합니다.

CSS 없이 HTML만 있으면 브라우저 기본 스타일이 적용된 밋밋한 페이지가 됩니다. CSS를 추가하면 세련된 디자인으로 바뀝니다.

JavaScript는 동작 (Behavior)

JavaScript는 페이지에 동적인 기능을 추가하는 프로그래밍 언어입니다. 버튼 클릭 처리, 데이터 불러오기, 화면 변경 등 사용자 상호작용을 구현합니다.

브라우저가 코드를 화면으로 그리는 과정

브라우저는 HTML, CSS, JS 파일을 받으면 다음 과정을 거쳐 화면에 그립니다.

1. **HTML 파싱(Parsing)**: HTML 코드를 읽어 **DOM(Document Object Model)** 트리 구조를 만듭니다.
2. **CSS 파싱**: CSS 코드를 읽어 스타일 규칙을 해석합니다.
3. **렌더 트리 구성**: DOM과 CSS를 결합하여 실제로 화면에 그릴 트리를 만듭니다.
4. **레이아웃(Layout)**: 각 요소의 크기와 위치를 계산합니다.
5. **페인팅(Painting)**: 픽셀 단위로 화면에 그립니다.
6. **JavaScript 실행**: JS 코드가 DOM을 변경하면 일부 단계를 다시 수행합니다.

이 전체 과정을 **렌더링(Rendering)**이라고 합니다.

팁: 브라우저마다 렌더링 엔진이 다릅니다. Chrome/Edge는 Blink, Firefox는 Gecko, Safari는 WebKit 엔진을 사용합니다. 대부분의 현대 브라우저에서 동일하게 동작하지만, 일부 최신 기능은 브라우저마다 지원 상황이 다를 수 있습니다.

코드로 확인하기

세 가지 언어가 어떻게 협력하는지 직접 보겠습니다. 아래 코드를 텍스트 에디터에 붙여넣고

`hello.html` 로 저장한 후 브라우저로 열어 보십시오.

```

<!DOCTYPE html>
<html lang="ko">
  <head>
    <meta charset="UTF-8">
    <title>세 언어의 협력</title>
    <!-- CSS: 스타일을 직접 style 태그에 넣은 예시 (나중에는 외부 파일로 분리합니다.) -->
    <style>
      h1 {
        color: #2563eb;          /* 글자 색을 파란색으로 */
        font-size: 2rem;        /* 글자 크기 */
      }
      button {
        background-color: #2563eb;
        color: white;
        padding: 8px 20px;
        border: none;
        border-radius: 6px;
        cursor: pointer;
      }
    </style>
  </head>
  <body>
    <!-- HTML: 페이지 구조 -->
    <h1 id="greeting">안녕하세요!</h1>
    <p>아래 버튼을 눌러 보세요.</p>
    <button id="changeBtn">인사 바꾸기</button>

    <!-- JavaScript: 동적 동작 -->
    <script>
      const btn = document.getElementById('changeBtn');
      btn.addEventListener('click', () => {
        document.getElementById('greeting').textContent = '반갑습니다, 웹 개발자';
      });
    </script>
  </body>
</html>

```

브라우저에서 파일을 열면 다음과 같이 표시됩니다.

안녕하세요!	← 파란색 큰 제목
아래 버튼을 눌러 보세요.	
[인사 바꾸기]	← 파란 배경의 버튼

"인사 바꾸기" 버튼을 클릭하면 제목이 "반갑습니다, 웹 개발자님!"으로 바뀝니다. HTML이 구조를 만들고, CSS가 디자인을 입히고, JavaScript가 클릭 동작을 처리했습니다.

줄별 코드 설명

줄	코드	설명
9	<code>color: #2563eb;</code>	h1 글자 색을 16진수 색상 코드로 파란색으로 지정합니다
14	<code>background-color: #2563eb;</code>	버튼 배경색을 파란색으로 지정합니다
24	<code><h1 id="greeting"></code>	JavaScript에서 식별할 수 있도록 <code>id="greeting"</code> 속성을 붙인 제목 태그입니다
26	<code><button id="changeBtn"></code>	클릭 이벤트를 연결할 버튼에도 <code>id</code> 를 부여합니다
30	<code>const btn = document.getElemen...</code>	<code>id</code> 로 버튼 요소를 찾아 <code>btn</code> 변수에 저장합니다
31	<code>btn.addEventListener('click', ...</code>	버튼에 클릭 이벤트 리스너를 등록합니다
32	<code>.textContent = ' ... '</code>	제목 요소의 텍스트 내용을 새 문장으로 교체합니다

주의: 이 예시는 세 언어의 협력을 한 파일에서 보여주기 위한 것입니다. 실제 프로젝트에서는 CSS와 JS를 별도 파일로 분리하는 것이 올바른 방식입니다. 이후 챕터에서 그 방법을 배웁니다.

절 요약

- HTML은 구조, CSS는 표현, JavaScript는 동작을 담당하여 역할이 분리됩니다.
- 브라우저는 HTML/CSS/JS를 받아 파싱 → 렌더 트리 → 레이아웃 → 페인팅 순서로 화면을 그립니다.
- 이 과정 전체를 렌더링이라고 합니다.

1.3 개발 환경 설정하기

도입

웹 페이지를 만들려면 두 가지만 있으면 됩니다. 코드를 작성할 **에디터(Editor)**와, 결과를 확인할 **브라우저(Browser)**입니다. 이 절에서는 무료이면서 강력한 VS Code를 설치하고, 파일 저장과 동시에 브라우저가 자동으로 새로고침되는 Live Server를 설정합니다.

핵심 개념 설명

코드 에디터란 무엇인가 (Code Editor)

코드 에디터(Code Editor)는 프로그래밍 코드를 작성하기 위해 최적화된 텍스트 편집 프로그램입니다. 일반 메모장과 달리 다음 기능을 제공합니다.

- **문법 강조(Syntax Highlighting)**: 태그, 속성, 값 등을 색상으로 구분하여 읽기 쉽게 합니다.
- **자동 완성(Auto Completion)**: `<h` 를 입력하면 `<h1>` , `<h2>` 등을 제안합니다.
- **에러 표시**: 잘못된 코드에 밑줄이나 경고를 표시합니다.
- **확장(Extension)**: 추가 기능을 설치하여 더욱 편리하게 사용할 수 있습니다.

VS Code 설치하기

VS Code(Visual Studio Code)는 Microsoft가 만든 무료 오픈소스 코드 에디터입니다. 가볍고 빠르며, 전 세계 개발자들이 가장 많이 사용하는 에디터입니다.

설치 방법:

1. 브라우저에서 <https://code.visualstudio.com> 으로 이동합니다.
2. 운영체제(Windows/macOS/Linux)에 맞는 설치 파일을 다운로드합니다.
3. 설치 파일을 실행하고 기본 옵션으로 설치를 완료합니다.
4. VS Code를 처음 열면 Welcome 탭이 표시됩니다.

팁: Windows 설치 시 "PATH에 추가"와 "Open with Code" 옵션을 체크하면 나중에 터미널에서 `code .` 명령으로 폴더를 바로 열 수 있어 편리합니다.

Live Server 확장 설치와 사용법

HTML 파일을 그냥 더블클릭하여 브라우저로 열면 파일을 수정해도 브라우저를 직접 새로고침해야 합니다. **Live Server** 확장은 파일이 저장될 때 브라우저를 자동으로 새로고침해 주어 개발 속도를 크게 높여 줍니다.

설치 방법:

1. VS Code 왼쪽 사이드바에서 네모 블록 아이콘(확장)을 클릭합니다. 단축키는 `Ctrl+Shift+X` (Windows) / `Cmd+Shift+X` (macOS)입니다.
2. 검색창에 `Live Server` 를 입력합니다.
3. Ritwick Dey가 만든 "Live Server"를 찾아 **Install** 버튼을 클릭합니다.
4. 설치가 완료되면 VS Code 하단 상태 표시줄에 **"Go Live"** 버튼이 나타납니다.

사용 방법:

1. HTML 파일을 열어 둔 상태에서 하단의 **"Go Live"** 버튼을 클릭합니다.
2. 브라우저가 자동으로 열리고 `http://127.0.0.1:5500/index.html` 같은 주소로 페이지가 표시됩니다.
3. VS Code에서 코드를 수정하고 저장(`Ctrl+S`)하면 브라우저가 즉시 새로고침됩니다.

정보: `127.0.0.1` 은 내 컴퓨터 자신을 가리키는 IP 주소입니다. Live Server가 내 컴퓨터를 임시 서버로 만들어 실제 웹 서비스처럼 파일을 제공하는 것입니다.

작업 폴더 구조 만들기

프로젝트마다 전용 폴더를 만드는 습관은 매우 중요합니다. 여러 파일이 한 폴더에 뒤섞이면 관리가 어려워집니다.

이 교재에서는 다음 구조를 기본으로 사용합니다.

my-web-project/	← 프로젝트 최상위 폴더
├── index.html	← 메인 페이지 (진입점)
├── css/	
│ └── style.css	← 스타일시트
├── js/	
│ └── main.js	← JavaScript 파일
└── images/	← 이미지 파일 모음

VS Code에서 폴더 열기:

1. **파일(File)** → **폴더 열기(Open Folder)** 를 선택합니다.
2. 작업할 프로젝트 폴더를 선택합니다.
3. 이후 VS Code의 탐색기(왼쪽 사이드바)에 폴더 구조가 표시됩니다.

코드로 확인하기

VS Code에서 새 파일을 만드는 전체 흐름을 단계별로 확인합니다.

1단계: 프로젝트 폴더 생성

먼저 바탕화면이나 원하는 위치에 **my-first-web** 폴더를 만듭니다.

2단계: VS Code로 폴더 열기

파일 → 폴더 열기 메뉴로 방금 만든 폴더를 선택합니다.

3단계: index.html 파일 생성

VS Code 탐색기에서 폴더 이름 옆의 **+** 아이콘을 클릭하거나 **파일 → 새 파일** 을 선택합니다.
파일명을 **index.html** 로 입력하고 엔터를 누릅니다.

4단계: HTML 기본 골격 자동 완성

빈 **index.html** 파일에서 **!** 를 입력한 후 **Tab** 키를 누릅니다. VS Code Emmet 기능이 자동으로 HTML 기본 골격을 생성합니다.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
  </head>
  <body>

  </body>
</html>
```

`lang="en"` 을 `lang="ko"` 로 바꾸고, `<title>` 태그 안의 `Document` 를 원하는 제목으로 바꿉니다.

팁: `!` + `Tab` 단축키는 VS Code의 Emmet 기능입니다. HTML 코드 작성 시간을 크게 줄여 주는 강력한 도구입니다. `h1` + `Tab` 을 입력하면 `<h1></h1>` 이 자동 완성되는 식으로 활용할 수 있습니다.

베스트 프랙티스: 파일명은 영문 소문자와 하이픈(-)만 사용합니다. 한글이나 공백이 포함된 파일명은 URL에서 문제를 일으킬 수 있습니다. 메인 페이지 파일명은 반드시 `index.html` 로 지정합니다. 서버는 폴더에 접근할 때 자동으로 `index.html` 을 찾습니다.

절 요약

- VS Code는 무료이면서 가장 인기 있는 코드 에디터입니다.
- Live Server 확장을 설치하면 저장 즉시 브라우저가 새로고침됩니다.
- 프로젝트마다 전용 폴더를 만들고 `index.html` 을 진입점으로 사용합니다.

1.4 첫 번째 웹 페이지 만들기

도입

드디어 실제로 코드를 작성할 시간입니다. 이 절에서는 지금까지 배운 내용을 바탕으로 첫 번째 HTML 파일을 완성하고 브라우저에서 직접 결과를 확인합니다. 완벽하지 않아도 됩니다. 화면에 내용이 뜨는 순간, 여러분은 웹 개발자가 됩니다.

핵심 개념 설명

HTML 파일(.html)이란

HTML 파일은 확장자가 `.html` 인 텍스트 파일입니다. 특별한 바이너리 포맷이 아니어서 메모장으로도 열고 편집할 수 있습니다. 다만 VS Code 같은 에디터를 사용하면 훨씬 편리합니다.

브라우저는 `.html` 파일을 받으면 처음부터 끝까지 순서대로 읽어 내려가며 화면을 그립니다.

기본 문서 골격 분해하기

모든 HTML 문서는 다음 구조로 시작합니다.

```
<!DOCTYPE html>
<html lang="ko">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>페이지 제목</title>
  </head>
  <body>
    <!-- 여기에 화면에 보이는 내용을 작성합니다 -->
  </body>
</html>
```

줄별 코드 설명

줄	코드	설명
1	<code><!DOCTYPE html></code>	이 파일이 HTML Living Standard(HTML5)로 작성되었음을 브라우저에 알립니다. 반드시 첫 줄에 위치해야 합니다
2	<code><html lang="ko"></code>	HTML 문서의 최상위 루트 요소입니다. <code>lang="ko"</code> 는 이 문서의 언어가 한국어임을 명시합니다
3	<code><head></code>	브라우저에게 필요한 메타 정보를 담는 영역입니다. 화면에 직접 표시되지 않습니다
4	<code><meta charset="UTF-8"></code>	문자 인코딩을 UTF-8로 지정합니다. 한글이 깨지지 않도록 반드시 포함해야 합니다
5	<code><meta name="viewport" ... ></code>	모바일 기기에서 화면 너비를 올바르게 처리하도록 설정합니다
6	<code><title></code>	브라우저 탭과 북마크에 표시될 페이지 제목을 지정합니다
7	<code></head></code>	head 영역을 닫습니다
8	<code><body></code>	화면에 실제로 표시될 콘텐츠를 담는 영역입니다
10	<code></body></code>	body 영역을 닫습니다
11	<code></html></code>	HTML 문서를 닫습니다

`<!DOCTYPE html>`

항목	내용
설명	HTML Living Standard 문서임을 선언하는 지시자(Doctype Declaration)입니다
위치	HTML 파일의 첫 번째 줄 (앞에 어떤 내용도 없어야 합니다)
목적	브라우저가 표준 모드(Standards Mode)로 페이지를 렌더링하도록 합니다. 생략하면 Quirks Mode가 되어 예상치 못한 렌더링 문제가 발생할 수 있습니다
사용 예시	<code><!DOCTYPE html></code> (대소문자 구분 없이 동작합니다)

`<meta charset="UTF-8">`

항목	내용
설명	문서의 문자 인코딩 방식을 지정하는 메타 태그입니다
속성	<code>charset</code> : 인코딩 방식 이름 (UTF-8 권장)
위치	<code><head></code> 내부의 첫 번째 자식으로 배치하는 것이 권장됩니다
중요성	생략하면 한글, 중국어, 일본어 등 비ASCII 문자가 깨져 표시됩니다
사용 예시	<code><meta charset="UTF-8"></code>

코드로 확인하기: Hello, World! 출력하기

프로그래밍을 처음 배울 때 관례적으로 "Hello, World!"를 출력하는 것부터 시작합니다. 웹에서도 같은 방식으로 시작해 보겠습니다.

이 코드는 브라우저 화면에 "Hello, World!"와 자기소개 텍스트를 표시하는 완전한 HTML 문서입니다.

```
<!DOCTYPE html>
<html lang="ko">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>나의 첫 웹 페이지</title>
  </head>
  <body>
    <h1>Hello, World!</h1>
    <p>저는 웹 개발을 배우고 있습니다.</p>
    <p>이것이 제 첫 번째 웹 페이지입니다!</p>
  </body>
</html>
```

브라우저 화면에는 다음과 같이 표시됩니다.

Hello, World!

← 굵고 큰 제목 (h1 기본 스타일)

저는 웹 개발을 배우고 있습니다.

← 일반 문단

이것이 제 첫 번째 웹 페이지입니다!

← 일반 문단

줄별 코드 설명

줄	코드	설명
1	<code><!DOCTYPE html></code>	HTML Living Standard 문서 선언입니다
2	<code><html lang="ko"></code>	한국어 문서임을 선언하는 루트 요소입니다
4	<code><meta charset="UTF-8"></code>	한글이 깨지지 않도록 UTF-8 인코딩을 선언합니다
5	<code><meta name="viewport" ... ></code>	모바일 화면에서 올바르게 표시되도록 뷰포트를 설정합니다
6	<code><title>나의 첫 웹 페이지</title></code>	브라우저 탭에 "나의 첫 웹 페이지"가 표시됩니다
9	<code><h1>Hello, World!</h1></code>	가장 큰 제목을 나타내는 h1 태그입니다. 브라우저가 굵고 크게 표시합니다
10	<code><p>저는 웹 개발을...</p></code>	문단(paragraph)을 나타내는 p 태그입니다. 위아래로 여백이 자동으로 생깁니다
11	<code><p>이것이 제 첫...</p></code>	두 번째 문단입니다. p 태그는 각 문단을 독립된 블록으로 분리합니다

브라우저에서 결과 확인하기

파일을 저장한 후 두 가지 방법으로 브라우저에서 확인할 수 있습니다.

방법 1: 파일 직접 열기

파일 탐색기에서 `index.html` 을 더블클릭하거나, 브라우저에서 **파일** → **파일 열기** 로 열 수 있습니다. 주소창에 `file:///C:/Users/...` 형태로 표시됩니다.

방법 2: Live Server 사용 (권장)

VS Code에서 파일을 열고 하단 "Go Live"를 클릭하면 `http://127.0.0.1:5500/index.html` 로 자동 접속됩니다. 코드를 수정하고 저장할 때마다 브라우저가 자동으로 갱신됩니다.

베스트 프랙티스: 가능하면 Live Server를 사용하십시오. 파일을 직접 열면 `file://` 프로토콜이 사용되는데, 일부 JavaScript 기능은 실제 HTTP 환경에서만 정상 동작합니다. Live Server는 실제 웹 서버 환경을 시뮬레이션하므로 더 정확한 결과를 볼 수 있습니다.

비교 테이블: 파일 직접 열기 vs. Live Server

구분	파일 직접 열기	Live Server
프로토콜	<code>file://</code>	<code>http://</code>
자동 새로고침	없음 (수동)	저장 시 자동
JavaScript 일부 기능	제한 있음	정상 동작
설정 필요 여부	불필요	VS Code 확장 설치 필요
권장 상황	간단히 확인할 때	개발 작업 전체

절 요약

- `<!DOCTYPE html>` 선언으로 HTML Living Standard 문서임을 명시합니다.
- `<head>` 에는 메타 정보, `<body>` 에는 화면에 보이는 콘텐츠를 작성합니다.
- `<meta charset="UTF-8">` 은 한글 깨짐을 방지하므로 반드시 포함합니다.
- Live Server를 사용하면 저장 즉시 브라우저에서 결과를 확인할 수 있습니다.

FAQ

Q1: HTML, CSS, JavaScript를 반드시 모두 배워야 합니까?

A1: 웹 페이지를 만들려면 기본적으로 세 언어 모두 필요합니다. 하지만 각자 역할이 분리되어 있어 단계별로 배울 수 있습니다. 이 교재처럼 HTML로 구조를 익히고, 이후 CSS로 꾸미기, 그 다음 JavaScript로 동적 기능을 추가하는 순서가 학습에 가장 효과적입니다.

Q2: 인터넷이 없어도 웹 페이지를 만들고 테스트할 수 있습니까?

A2: 네, 가능합니다. HTML 파일을 작성하고 브라우저에서 직접 열거나 Live Server를 실행하면 인터넷 연결 없이도 내 컴퓨터 안에서 완전히 동작합니다. 인터넷은 다른 사람이 만든 웹 서버에 접속하거나, 외부 API를 호출할 때만 필요합니다.

Q3: `<!DOCTYPE html>` 없이도 페이지가 보이던데, 꼭 써야 합니까?

A3: 대부분의 경우 페이지가 표시되기는 합니다. 하지만 DOCTYPE을 생략하면 브라우저가 "Quirks Mode(비표준 모드)"로 동작하여 예상치 못한 레이아웃이나 스타일 오류가 발생할 수 있습니다. 특히 박스 모델 계산이 달라져 CSS 학습에 방해가 됩니다. 항상 첫 줄에 `<!DOCTYPE html>` 을 작성하는 습관을 들이십시오.

Q4: VS Code 외에 다른 에디터를 사용해도 됩니까?

A4: 물론입니다. Sublime Text, Atom, Notepad++ 등 다양한 에디터를 사용할 수 있습니다. 심지어 메모장으로도 HTML을 작성할 수 있습니다. 하지만 VS Code는 무료이면서 강력한 기능을 제공하고, 이 교재의 설명이 VS Code를 기준으로 작성되어 있으므로 가능하면 VS Code를 사용하기를 권장합니다.

장 요약

이번 장에서는 웹 프론트엔드 개발의 출발점이 되는 핵심 개념들을 살펴보았습니다. 브라우저가 URL을 입력받으면 HTTP 요청으로 서버에서 파일을 받아오고, HTML·CSS·JavaScript 세 언어의 협력으로 화면이 그려지는 원리를 이해했습니다. VS Code와 Live Server로 개발 환경을 구축하고, 첫 번째 HTML 파일을 직접 브라우저에서 실행해 보았습니다. 이제 여러분은 웹 개발의 첫 발을 내딛었습니다.

핵심 키워드

웹 동작 원리 , 클라이언트-서버 , HTTP , URL , 브라우저 렌더링 , HTML , CSS , JavaScript , VS Code , Live Server , DOCTYPE , UTF-8

다음 장 미리보기

다음 장에서는 HTML 문서의 구조를 더 깊이 파고듭니다. 태그, 요소, 속성의 정확한 개념을 익히고 제목, 단락, 목록으로 의미 있는 문서를 직접 작성해 봅니다.

✂ 실습 프로젝트 (Hands-on Project)

나의 첫 웹 페이지 프로젝트

이 장에서 배운 개념을 실무 시나리오에 적용하는 프로젝트입니다.

초급: 나의 첫 웹 페이지 만들기

시나리오

여러분은 이제 막 웹 개발을 시작한 입문자입니다. 자신을 소개하는 첫 번째 웹 페이지를 만들어서 친구나 가족에게 공유하려고 합니다. Hello, World! 전통을 따라 첫 페이지를 완성하고, 이름과 좋아하는 한 문장을 담아 브라우저에서 실행해 봅니다. 이 경험을 통해 HTML 기본 골격과 브라우저 실행 방법에 자신감을 얻습니다.

학습 목표

- HTML 기본 골격(DOCTYPE, html, head, body)을 올바르게 작성할 수 있습니다
- h1, p 태그로 제목과 문단을 구성할 수 있습니다
- 파일을 브라우저로 열거나 Live Server로 실행할 수 있습니다

진행 방법

1. `project_starter.html` 을 열고 TODO 주석을 찾습니다
2. 각 TODO를 이 장에서 배운 개념으로 완성합니다
3. 완성 후 브라우저에서 열어 결과를 확인합니다

실행 방법

브라우저에서 `project_starter.html` 파일을 직접 열거나,
VS Code에서 파일을 열고 하단 "Go Live" 버튼을 클릭합니다.

중급: 나를 소개하는 인터랙티브 페이지

시나리오

초급 프로젝트에서 만든 자기소개 페이지를 발전시킵니다. 이번에는 CSS로 페이지를 꾸미고, JavaScript로 인터랙티브 기능(버튼 클릭 시 메시지 변경)을 추가합니다. HTML/CSS/JavaScript 세 언어가 한 페이지에서 협력하는 경험을 통해 각 언어의 역할을 몸으로 익힙니다.

학습 목표

- CSS를 style 태그 안에 작성하여 페이지 디자인을 적용할 수 있습니다
- JavaScript로 버튼 클릭 이벤트를 처리하고 페이지 내용을 동적으로 변경할 수 있습니다
- HTML/CSS/JS 세 언어의 역할 분리를 직접 체험할 수 있습니다

진행 방법

1. `project_advanced_starter.html` 을 열고 TODO 주석을 찾습니다
2. 초급에서 연습한 HTML 골격 지식을 바탕으로 CSS와 JavaScript 부분을 완성합니다
3. 완성 후 브라우저에서 열어 버튼 클릭 동작을 확인합니다

실행 방법

브라우저에서 `project_advanced_starter.html` 파일을 직접 열거나, VS Code에서 파일을 열고 하단 "Go Live" 버튼을 클릭합니다.

확장 아이디어

- CSS로 배경색, 글자 색, 여백을 더 꾸며 나만의 스타일을 만들어 보세요
- 버튼을 추가하여 다른 메시지를 표시하는 기능을 구현해 보세요
- h2 소제목을 추가하여 "좋아하는 것", "꿈" 등의 섹션을 더 만들어 보세요

▶  `project_starter.html` (스타터 코드 - 직접 완성해보세요)

▶  `project_complete.html` (완성 코드)

▶  project_advanced_starter.html (스타터 코드 - 직접 완성해보세요)

▶  project_advanced_complete.html (완성 코드)